Jarvis MVP - arquitetura honesta (hoje + futuro)

Cores = existe hoje. Cinza claro = vai ser criado. Setas tracejadas = opcional/condicional.

LOCAL (PC)

Voce
pede algo
Lat: humana

comunicacao/protocolo.py
formato das mensagens

Interface (entrada/saida)
texto e voz (STT/TTS) + UI/painel

Voz (entrada)

- **entrada/preflight.py** — confere microfone
- **entrada/app.py** — inicia voz + texto
- **entrada/audio_utils.py** — normaliza PCM
- **voz/vad.py** — detecta quando ha fala
- **entrada/stt.py** — voz vira texto; Rust + debounce
- **rust/jarvis_audio** — trim/silencio em Rust
- **voz/speaker_verify.py** — confere se e sua voz
- **voz/adapters/speaker_resemblyze** — ponte p/ verificador
- **voz/adapters/wakeword_text.py** — filtra wake word (texto)
- **voz/adapters/wakeword_porcupine.py** — detecta wake word (audio)

Texto/UI/saida

- **entrada/chat_ui.py** — caixinha para digitar
- **entrada/gui_panel.py** — painel + cancelar
- **entrada/shortcut.py** — atalho global do chat
- **entrada/followup.py** — janela + limite
- **comunicacao/chat_inbox.py** — ponte p/ comandos
- **voz/tts.py** — texto vira voz
- **comunicacao/chat_log.py** — diario do chat
- **voz/adapters/base.py** — contratos da voz
- **DOC_INTERFACE** — docs da interface

Deps da interface (entrada/saida)
sounddevice + numpy + scipy (+portaudio/alsa)
webrtcvad + silero-vad (VAD) + AEC simples + faster-whisper (STT)
piper/espeak-ng + aplay (TTS)
resemblyzer + cryptography (locutor)
pynput + watchdog + tkinter (UI/atalho)
pvporcupine/openwakeword (wake word audio)
py-spy + psutil + nvidia-smi (perf)

Testes da interface (entrada/saida)
test_preflight_* - headless/perfis/probe/atalho
test_audio_utils.py - PCM/bytes
test_chat_inbox.py - entrada por arquivo
test_chat_log.py - saida/log
test_chat_ui_interface.py - UI escreve no inbox
test_gui_panel_interface.py - painel envia comando + cancelar
test_shortcut_interface.py - atalho global/Wayland
test_app_voice_interface.py - CLI voz/loop/chat
test_stt_flow.py + test_stt_metrics.py + teste_stt_gap_latency.py + test_stt_whisper_kwargs.py - streaming/metricas/latencia/params
test_stt_streaming_controls.py + test_stt_partials.py + test_stt_silero_deactivity.py - earl
test_stt_rust_trim.py - trim Rust
test_stt_filters.py - wake word
test_stt_capture_config.py + test_stt_vad_env.py - device/SR/env VAD
test_audio_resample.py - reamostra 44k->16k
test_vad_pre_roll.py + test_vad_metrics.py - VAD pre-roll/metricas
test_vad_streaming_interface.py + test_aec_interface.py - VAD/AEC
test_followup_mode.py - follow-up
test_voice_adapters.py - adapters de voz
test_speaker_verify_interface.py - verif. de locutor
test_voice_max_seconds_env.py - max_seconds
test_tts_interface.py - TTS piper/espeak
test_bench_interface.py + scripts/bench_interface.py - lat/CPU/RAM
scripts/auto_voice_bench.py - voz real (silencio + ruido)
pytest -q testes/ (suite completa)

Performance atual (bench 2026-01-12)
Lat (voz 4s): STT 5.7-8.6s | VAD ~1ms
RSS: ver EVOLUCAO_PERFOMACE.MD
Disk 50-200MB (+log 1-10MB)

Executor local (gatekeeper)
junta tudo, pede confirmacao
gate + budget + kill switch; inclui loop S3 (GUI)
codigo: jarvis/cerebro/orchestrator.py + jarvis/agent_s3/*
CPU 5-15%, RAM 200-500MB, Lat 20-80ms

Regras de seguranca
bloqueia apps/dados + classifica texto
codigo: jarvis/seguranca/*
CPU <2%, RAM 20-60MB, Lat <10ms

Confirmacao humana
voz/tecla antes de agir
codigo: jarvis/cerebro/orchestrator.py
CPU <2%, RAM 20-60MB, Lat: humana

Validacao de saida
validacao visual/OCR da tela
codigo: jarvis/validacao/validator.py
CPU 2-8%, RAM 50-120MB, Lat 0.05-6s (Wayland se

Mascara de privacidade
remove dados sensiveis
codigo: jarvis/seguranca/privacy.py
CPU 5-25%, RAM 80-200MB, Lat 50-200ms

Maos (acoes) - hoje
mouse/teclado/web + drag/overwrite (S3)
hoje: Python em jarvis/acoes/desktop.py + web.py (
CPU 5-20%, RAM 80-200MB, Lat 50-200ms

Olhos (vision/OCR)
confere tela + grounding S3
hoje: rust/jarvis_vision + jarvis/agent_s3/grounding.py
CPU 30-80%, RAM 300-800MB
Disk 200-600MB, Lat 0.3-1.2s

Tela e apps
onde o Jarvis clica
CPU/RAM/GPU: n/a, Lat: depende app

Cache local + memoria
episodios + conhecimento + procedures
procedures.db + memory.sqlite3 (FTS + embeddings BLOB)
codigo: jarvis/memoria/* (dedup hash + cache LRU)
CPU <2%, RAM 30-80MB, Disk 50MB-2GB

Cerebro local (LLM)
endpoint OpenAI-compat self-hosted (LLM
codigo: jarvis/cerebro/llm.py
CPU 200-350%, RAM 6-10GB, Disk 6-12GB, Lat 2-8s

Guidance humano (manual)
Navegador: copia/cola se usuario quiser
sem API externa; so apoio humano opcional
CPU 2-10%, RAM 200-600MB, Lat humana

Bridge Rust (futuro)
daemon JSON-RPC local
futuro: rust/jarvis_bridge
CPU 1-5%, RAM 30-100MB, Lat 1-5ms

Maos Rust (futuro)
input Wayland + AT-SPI
futuro: rust/jarvis_actions
CPU 5-15%, RAM 50-150MB, Lat 20-80ms

Total PC (estimado + capacidade)
Base: CPU 40-120%, RAM 1.2-2.2GB
Com LLM local: CPU 200-350%, RAM 6-10GB
Disk 0.5-2GB (+6-12GB com LLM), Lat 0.2-8s
Capacidade atual (medir): CPU ___, RAM ___GB, GPU ___GB, Disk ___GB

VPS (CPU) - SELF-HOSTED

Orquestrador no VPS (futuro)
recebe pedido do PC e planeja
codigo: jarvis/cerebro/orchestrator.py (vai rodar no VPS)
CPU 15-40%, RAM 300-800MB, Lat 20-100ms

LLM base Qwen2.5-7B-Instruct (futuro)
servico llama.cpp + GGUF Q4
CPU 200-350% (4 vCPU), RAM 6-10GB
Disk 6-12GB, Lat 2-8s por plano

Gate de confianca + escada
escolhe: Qwen2.5 local (Mock fallback)
futuro: jarvis/cerebro/llm.py
CPU 5-15%, RAM 100-300MB, Lat 20-100ms

Memoria principal (futuro)
SQLite + FTS + vetores (384)
hoje: local FTS+vetores; VPS futuro
CPU 5-20%, RAM 200-600MB, Disk 1-10GB
Lat 20-200ms

RAG + resumo (futuro)
busca lembrancas antes de responder
futuro: jarvis/memoria/*
CPU 10-40%, RAM 300-800MB, Lat 0.1-0.8s

Telemetria + budget diario
corta gasto ao passar limite
futuro: jarvis/telemetria/*

CPU <5%, RAM 50-150MB, Disk 100MB-1GB

Exportacao para Drive (futuro)
jsonl.zst + manifesto via rclone
CPU <10% (batch), RAM 50-200MB
Disk temp 1-5GB, Lat: batch (min)

Pipeline de experimentos (futuro)
testa antes de mudar e pede aprovacao
futuro: rust/jarvis_experiments
CPU 20-80%, RAM 0.5-2GB, Lat: offline

Total VPS (estimado, com Qwen2.5)
CPU 250-400% (2.5-4 vCPU), RAM 8-14GB
Disk 10-25GB, Lat 2-8s por plano
Capacidade plano: 4 vCPU, 16GB RAM, 160GB SSD

SERVIDOR SELF-HOSTED (VPS/GPU OPCIONAL)

Cérebro Open Source (Qwen2.5 + LoRA)
llama.cpp / vLLM + GGUF, PEFT e bitsandbytes
codigo: jarvis/cerebro/llm.py + Documentos/PLANO_UNICO.md
CPU/RAM: 200-400% (4vCPU), 10-18GB, Lat 1.5-6s

Quantização responsiva + LoRA
GGUF/GPTQ/AWQ + peft.merge + bitsandbytes
documento: jarvis/ops/quantization.md (futuro)
Lat 0.5-3s (varia conforme o formato)

Treino incremental
captura logs → curadoria → LoRA → validação
scripts: jarvis/aprendizado/ (futuro)
Lat backend: batch, varia por ciclo

Servidor GPU/VPS
Hetzner CX32 + GPU NVidia 16GB ou similar
Disk 160GB, Lat 1-4s
monitor: systemd + Prometheus simples

Arquivo frio + exportação
jsonl.zst + manifesto via rclone
backup mensal, latência batch (min)
disk 1TB+, lat: obediente

Total Servidores dedicados
CPU 250-450% (4vCPU+GPU), RAM 14-20GB
Lat 1-6s por plano, Disk 160GB+
GGUF + LoRA → menor consumo, escala com GPU

Conexões

- Mock + browser manual (fallback)
- WebSocket/gRPC + TLS + auth (futuro obrigatorio)
- protocolo: jarvis/comunicacao/protocolo.py
- STT local/auto (faster-whisper)
- HTTP self-hosted (OpenAI compat, LLM + grounding)
- Quantização + LoRA (futuro)
- Backup JSONL/manifesto via rclone

Legenda

Cores fortes = existe hoje. Cinza = futuro. Setas coloridas = fluxo; tracejadas = opcional/condicional.

Metricas por balao: CPU, RAM, GPU, DISK, LAT (latencia).

Termos: LLM = IA que escreve texto; OCR = le texto da tela; RAG = busca memoria antes de responder; VPS = computador na nuvem.
S3 = agente GUI com grounding (jarvis/agent_s3).

Obs: pesos do diagrama refletem o baseline 2026-01-10 em `Documentos/DOC_INTERFACE/EVOLUCAO_PERFOMACE.MD`.

Deps opcionais: sounddevice+numpy+faster-whisper (STT), webrtcvad/silero-vad (VAD), piper/espeak (TTS),
xdotool/wtype/ydotool/pyautogui (acoes), Playwright (web), Pillow+pytesseract+tesseract (OCR), sentence-transformers (embeddings), pynput (aprendizado), tkinter (chat), gguf-tools/peft (LLM).